

Code examples

Payment processing:

```
module Commands
  module PaymentOrders
    module Helpers
      class ProcessEgress < Trailblazer::Operation

        step Wrap(->((ctx, _), *, &block) {
          ActiveRecord::Base.transaction do
            ctx[:model] =
              PaymentOrder.preload(in: :currency, out: :currency).lock
                .find_by(id: ctx[:payment_order_id] || ctx[:model].id)
            return false unless ctx[:model]
            block.call
          end
        }) {
          step ->(_ , model:, **) { model.may_process? }
          pass :calculate_charges
          pass ->(options, model:, **) { options[:account] = model.in }
          pass ->(options, model:, charges_data:, **) { options[:amount] = model.amount + charges_data.total }
          step ->(_ , amount:, **) { amount > 0 }
          step Subprocess(CheckBalance)
          step :create_transactions!
          step :create_charges!
          fail ->(_ , **) { raise ActiveRecord::Rollback }
        }
        fail ->(_ , model:, **) { model&.reload }

        pass :send_to_aml
        pass :enqueue_process_transaction

        fail :mark_as_fail!
        pass Subprocess(Notify)
        pass :send_events
        fail ->(_ , params) { Services::LogCommand.(command_klass: self.class, params: params) }

        def calculate_charges(options, model:, **)
          return unless model.in.client_account&.client

          options[:charges_data] = Services::Commission.new(
            client: model.in.client_account.client,
            client_account: model.in.client_account,
            payment_order: model,
            product_code: Services::PaymentProductCode.for_send(model.out)
          )
          return unless options[:charges_data].any?

          model.in_commission = options[:charges_data].detailed_float
        end

        def create_transactions!(options, model:, **)
          conditions = {
            same_currency: model.in.currency.value == model.out.currency.value,
            same_bank_entity: model.in.bank_entity_id == model.out.bank_entity_id
          }

          case conditions
          in same_currency: true, same_bank_entity: true
            create_simple_transaction(options, model)
          in same_currency: false, same_bank_entity: true
            create_fx_transaction(options, model)
          in same_bank_entity: false

```

```

        create_cross_entity_transaction(options, model)
      else
        false
      end
    end
  end

  def create_charges!(_, model:, transaction:, charges_data:, **)
    charges_data.detailed.each do |item|
      charge_amount = (model.out.currency.kind.to_sym == :currency) ? item[:value].floor(2) : item[:value]
      TransactionItem.create(
        in: model.in,
        out: get_service_account(model.in.bank_entity_id, model.in.currency.value, item[:service_account_code]),
        amount: charge_amount,
        charge_for: transaction,
        comment: "Charge for transfer ##{model.id} - #{item[:display_name]}"
      )
    end
  end

  def create_simple_transaction(options, model)
    options[:transaction] = TransactionItem.create(
      in: model.in,
      out: model.out,
      amount: model.amount,
      comment: model.comment
    )
    update_payment_order(options, model)
  end

  def create_fx_transaction(options, model)
    create_separated_transaction(options, model, :exchange)
  end

  def create_cross_entity_transaction(options, model)
    if model.in.bank_entity.provider_code&.to_sym != :arival_us ||
       model.out.bank_entity.provider_code&.to_sym != :arival_us
      return false
    end
    create_separated_transaction(options, model, :cross_entity)
  end

  def create_separated_transaction(options, model, service_account_code)
    income_intermediate_account =
      get_service_account(model.in.bank_entity_id, model.in.currency.value, service_account_code)

    expense_intermediate_account =
      get_service_account(model.out.bank_entity_id, model.out.currency.value, service_account_code)
    return false if income_intermediate_account.nil? || expense_intermediate_account.nil?

    converted_amount =
      Services::Rates.convert(
        source_currency: model.in.currency.value,
        dest_currency: model.out.currency.value,
        volume: model.amount
      )

    if model.out.currency.kind.to_sym == :currency
      converted_amount = converted_amount.floor(2)
    end

    options[:transaction] = TransactionItem.create!(
      in: model.in,
      out: income_intermediate_account,
      amount: model.amount,
      comment: model.comment
    )

    TransactionItem.create!(

```

```

        in: expense_intermediate_account,
        out: model.out,
        amount: converted_amount,
        comment: model.comment,
        related_for: options[:transaction]
    )

    update_payment_order(options, model)
end

def update_payment_order(options, model)
  model.process
  model.transaction_item = options[:transaction]
  model.save
end

def send_to_aml(_, model:, **)
  if model.should_aml_check?
    model.send_to_aml!
    aml_client = Services::AmlClient.new
    aml_client.aml_check(model)
  else
    model.skip_aml!
  end
end

def enqueue_process_transaction(_, model:, **)
  if model.waiting_for_approve?
    Helpers::TryProcessTransactionsWithoutApprove.(model: model)
  end
end

def send_events(_, model:, **)
  pusher = Services::EventPusher.new
  model.display_transactions.each do |t|
    pusher.push('payment_created', Serializers::DisplayTransaction.render_as_hash(t))
  end
end

def get_service_account(bank_entity_id, currency, service_code)
  result =
    ServiceAccounts::FindOrCreate.(
      bank_entity_id: bank_entity_id,
      service_code: service_code,
      currency: currency
    )
  result.success? ? result[:model].account : nil
end

def mark_as_fail!(options, model:, **)
  model.fail! if model.may_fail?
end

end
end
end
end

```

Client model definition with encryption, blind indexes and state machine:

```

class Client < ApplicationRecord
  extend AaCipher::ActiveRecord
  include AASM

  KINDS = %i[personal entity]

```

```

belongs_to :bank_entity
belongs_to :tariff
belongs_to :country, class_name: 'DirectoryItem'
belongs_to :limit_tier

has_many :client_accounts, dependent: :destroy
has_many :external_links, as: :subject, dependent: :destroy
has_many :synapse_ext_ach_accounts, dependent: :destroy
has_many :synapse_ext_wire_inter_accounts, dependent: :destroy
has_many :synapse_ext_wire_local_accounts, dependent: :destroy
has_many :synapse_users, dependent: :destroy
has_many :recurring_fee_event_clients, dependent: :destroy

encrypted_attr :display_name, :str, index: true
encrypted_attr :entity_name, :str
encrypted_attr :address, :str
encrypted_attr :aid_url, :str
encrypted_attr :first_name, :str
encrypted_attr :last_name, :str
encrypted_attr :id_no, :str
encrypted_attr :id_type, :str
encrypted_attr :dob, :time
encrypted_attr :tax_id, :str
encrypted_attr :company_registration_number, :str
encrypted_attr :extended_data, :json, default: {}

blind_index :display_name, expression: ->(t) { t&.downcase }
blind_index :entity_name, expression: ->(t) { t&.downcase }
blind_index :address, expression: ->(t) { t&.downcase }
blind_index :aid_url, expression: ->(t) { t&.downcase }
blind_index :first_name, expression: ->(t) { t&.downcase }
blind_index :last_name, expression: ->(t) { t&.downcase }
blind_index :id_no, expression: ->(t) { t&.downcase }
blind_index :id_type, expression: ->(t) { t&.downcase }
blind_index :tax_id, expression: ->(t) { t&.downcase }
blind_index :company_registration_number
blind_index :dob

aasm(column: :state) do
  state :created, initial: true
  state :active
  state :rejected
  state :on_hold
  state :blocked

  event :activate do
    transitions from: [:created, :on_hold], to: :active
  end

  event :reject do
    transitions from: [:created, :on_hold], to: :rejected
  end

  event :to_on_hold do
    transitions from: [:created, :active], to: :on_hold
  end

  event :block do
    transitions from: :active, to: :blocked
  end

  event :unblock do
    transitions from: :blocked, to: :active
  end

  event :reactivate do
    transitions from: :on_hold, to: :active
  end
end

```

```

end

def aid_data
  @aid_data ||= Services::AidData.new(self)
end
end

```

Currency converter:

```

module Services
  module Rates
    class << self

      INTERMEDIATE_CURRENCY = 'USD'.freeze

      def get_rate(source_currency:, dest_currency:, date: nil)
        rate, _ = get_rate_and_spread(source_currency, dest_currency, date)
        rate
      end

      def convert(source_currency:, dest_currency:, volume:, spread: nil, date: nil)
        rate, def_spread = get_rate_and_spread(source_currency, dest_currency, date)
        spread ||= def_spread
        volume * rate * (1 - spread)
      end

      private

      def get_rate_and_spread(source_currency, dest_currency, date=nil)
        source_currency = source_currency.to_s.upcase
        dest_currency = dest_currency.to_s.upcase
        return [1, 0] if source_currency == dest_currency
        arr = [source_currency, dest_currency]
        direct_rate = find_rate(arr, arr, date)
        if direct_rate
          spread = direct_rate.spread
          if direct_rate.base == source_currency
            rate = direct_rate.value
          elsif direct_rate.quote == source_currency
            rate = 1 / direct_rate.value
          else
            raise
          end
        else
          r1 = find_rate(source_currency, INTERMEDIATE_CURRENCY, date)
          r2 = find_rate(dest_currency, INTERMEDIATE_CURRENCY, date)
          rate = r1.value / r2.value
          spread = r1.spread + r2.spread
        end
        [rate, spread]
      end

      def find_rate(base, quote, date)
        if date
          rate = Rate.where(quote: quote, base: base).where('created_at < ?', date).order(created_at: :desc).first
          rate || find_rate(base, quote, nil)
        else
          Rate.where(quote: quote, base: base).order(created_at: :desc).first
        end
      end

    end
  end
end

```

Client Drafts controller:

```
module Controllers
  class ClientDraftsController < ApplicationController

    FIELDS = %i[kind tariff_id country_id primary_currency_id display_name entity_name
               first_name last_name dob special_activities company_registration_number]

    post '' do
      result = Commands::Api::ClientDrafts::Create.(
        user: current_user,
        attributes: params.slice(*FIELDS)
      )

      response_payload = {
        success: result.success?,
        result: result.success? ? Serializers::ClientDraft.render_as_hash(result[:model]) : nil,
        error: result[:error]
      }
      response_payload.to_json
    end

    get '' do
      Serializers::ClientDraft.render(scope)
    end

    get '/:id' do
      Serializers::ClientDraft.render(resource)
    end

    put '/:id' do
      result = Commands::Api::ClientDrafts::Update.(
        user: current_user,
        client_draft_id: params.fetch(:id),
        attributes: params.slice(*FIELDS)
      )

      response_payload = {
        success: result.success?,
        result: result.success? ? Serializers::ClientDraft.render_as_hash(result[:model]) : nil,
        error: result[:error]
      }
      response_payload.to_json
    end

    put '/:id/sign' do
      result = Commands::Api::ClientDrafts::DirectSign.(
        user: current_user,
        client_draft_id: params.fetch(:id)
      )

      response_payload = {
        success: result.success?,
        result: result.success? ? Serializers::ClientDraft.render_as_hash(result[:model]) : nil,
        error: result[:error]
      }
      response_payload.to_json
    end

    get '/:id/document_signs' do
      Serializers::DocumentSign.render(resource.document_signs)
    end

    post '/:id/document_signs/:doc_id/url' do
      result = Commands::Api::ClientDrafts::CreateDocumentSignUrl.(
        client_draft_id: params[:id],
        document_sign_id: params[:doc_id]
      )
    end
  end
end
```

```

    )

    {
      success: result.success?,
      result: result[:result],
      error: result[:error]
    }.to_json
  end

  get '/:id/aid_verifications' do
    Serializers::AidVerification.render(resource.aid_verifications.ordered)
  end

  get '/:id/aid_verifications/:verification_id' do
    form = resource.aid_verifications.find(params.fetch(:verification_id))
    Serializers::AidVerification.render(form)
  end

  get '/:id/aid_verifications/:verification_id/redirect' do
    result = Commands::Api::AidVerifications::BuildUrl.call(
      subject: resource,
      aid_verification_id: params[:verification_id]
    )
    response_payload = {
      success: result.success?,
      result: result[:redirect_url],
      error: result[:error]
    }
    response_payload.to_json
  end

  private

  def scope
    current_user.client_drafts.ordered
  end

  def resource
    scope.find_by(id: params.fetch(:id))
  end

end
end

```